



Level 3 – Advanced

Duração estimada: 5 meses (20 semanas · 5-10h/semana)

Pré-requisito: Nível 2 completo

Foco: Microsserviços, mensageria, observabilidade, performance, K8s

Projetos: Sistema de Pagamentos · Servidor HTTP em C++



Skills ao final do nível

- [] **Message Brokers:** RabbitMQ (exchanges, DLQ, RPC) e Kafka (tópicos, partições, consumer groups)
 - [] **Event-Driven Architecture:** Event sourcing, saga pattern (coreografada/orquestrada)
 - [] **Microsserviços:** Service discovery, API Gateway, comunicação síncrona/assíncrona
 - [] **gRPC:** Protocol Buffers, unary/streaming, performance vs REST
 - [] **GraphQL:** Schema, resolvers, DataLoader (N+1 problem), subscriptions
 - [] **Observabilidade:** Métricas (Prometheus), tracing (OpenTelemetry), logs estruturados
 - [] **Kubernetes:** Pods, Services, Deployments, ConfigMaps, probes, HPA
 - [] **Performance:** Load testing (k6), query profiling, connection pooling, benchmarking
 - [] **C++:** Servidor HTTP multi-thread, thread pool, sockets, memory-mapped files
 - [] **Circuit Breaker:** Proteger dependências externas, fallback, half-open state
 - [] **Streaming:** Streaming responses, file upload com streams, backpressure
 - [] **HTTP/2 Multiplexing:** Conexões simultâneas, server push, HPACK
-



Semana a Semana

SEMANA 1 – RabbitMQ: Mensageria Assíncrona

 **Teoria (3h)** - Leia: [RabbitMQ – Get Started \(tutoriais 1-5\)](#) - Assista: [Hussein Nasser – Message Queues Explained](#) - Estude: exchanges (direct, topic, fanout), DLQ, publisher confirms, prefetch

Prática (5h)

```
// Producer: publica evento de pedido
await channel.assertExchange('orders', 'topic', { durable: true });
channel.publish('orders', 'order.created', Buffer.from(JSON.stringify(order)));

// Consumer: processa pagamento
const queue = await channel.assertQueue('payments', { durable: true });
await channel.bindQueue(queue.queue, 'orders', 'order.created');
channel.consume(queue.queue, async (msg) => {
  try {
    await processPayment(JSON.parse(msg.content.toString()));
    channel.ack(msg);
  } catch (err) {
    // Rejeita e envia para DLQ
    channel.nack(msg, false, false);
  }
});

// DLQ (Dead Letter Queue)
await channel.assertQueue('payments.dlq', { durable: true });
// Config: arguments: { "x-dead-letter-exchange": "orders.dlx" }
```

 **Exercício:** Crie 3 exchanges (orders, notifications, logs) + 3 consumers + 2 DLQs. Simule falha e veja a mensagem cair na DLQ.

SEMANA 2 – Event-Driven Architecture + Saga Pattern

 **Teoria (4h)** - Leia: [Martin Fowler – Event Sourcing](#) - Leia: [Saga Pattern \(Microsoft\)](#) - Estude: coreografada vs orquestrada, compensação, consistência eventual

Prática (5h)

```
// Saga coreografada (cada serviço publica/escuta eventos)

// Order Service
// 1. Cria pedido (status: pending)
// 2. Publica "order.created"

// Payment Service
// 1. Escuta "order.created"
// 2. Processa pagamento
// 3. Publica "payment.completed" ou "payment.failed"

// Inventory Service
// 1. Escuta "payment.completed"
// 2. Reserva estoque
// 3. Publica "inventory.reserved" ou "inventory.failed"

// Compensação: se "inventory.failed"
// → Payment Service escuta e faz estorno (publica "payment.refunded")
// → Order Service escuta "payment.refunded" e cancela pedido
```

 **Exercício:** Implemente o fluxo de 3 serviços com RabbitMQ + saga. Teste o cenário de falha no estoque.

SEMANA 3 – gRPC + Protocol Buffers

 **Teoria (3h)** - Leia: [gRPC Docs – Core Concepts](#) - Leia: [Protocol Buffers – Language Guide](#) - Estude: unary vs server-streaming vs client-streaming vs bidirectional

 **Prática (5h)**

```
service FraudDetection {
  rpc Analyze (Transaction) returns (FraudResult);
  rpc StreamAlerts (AlertFilter) returns (stream Alert);
}

message Transaction {
  string id = 1;
  double amount = 2;
  string card_token = 3;
  string ip_address = 4;
}

message FraudResult {
  bool approved = 1;
  double risk_score = 2;
  string reason = 3;
}
```

```
// Server (Node/TS)
const server = new grpc.Server();
server.addService(FraudDetectionService, {
  Analyze: (call, callback) => {
    const { amount, ip_address } = call.request;
    const risk = calculateRisk(amount, ip_address);
    callback(null, { approved: risk < 0.7, risk_score: risk });
  },
  StreamAlerts: (call) => {
    setInterval(() => {
      call.write({ alertId: uuidv4(), severity: 'high', message: '...' });
    }, 5000);
  }
});
```

 **Exercício:** Compare performance de gRPC vs REST (JSON) com k6. Documente os resultados.

SEMANA 4 – Microsserviços: API Gateway + Service Discovery

 **Teoria (3h)** - Leia: [API Gateway Pattern](#) - Leia: [Service Discovery \(Martin Fowler\)](#)

 **Prática (5h)**

```
// API Gateway (Node/TS + http-proxy-middleware)
app.use('/api/users', proxy({ target: 'http://user-service:3001' }));
app.use('/api/orders', proxy({ target: 'http://order-service:3002' }));
app.use('/api/payments', proxy({ target: 'http://payment-service:3003' }));

// Com autenticação centralizada
app.use('/api/*', authMiddleware);
app.use('/api/admin/*', requireRole('admin'));

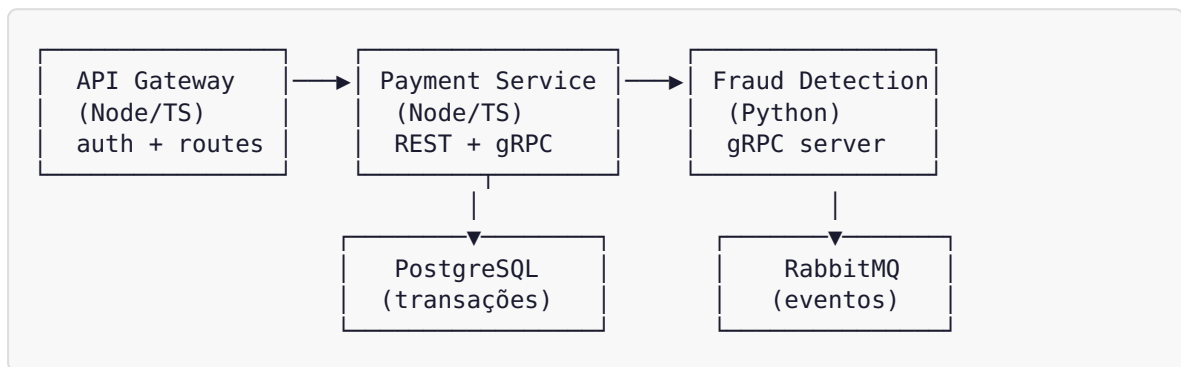
// Rate limiting por serviço
app.use('/api/payments', rateLimit({ windowMs: 60000, max: 30 }));
```

 **Exercício:** Crie 3 microsserviços simples + API Gateway com auth. Teste roteamento e rate limiting.

SEMANAS 5-8 – Projeto: Sistema de Pagamentos (Microsserviços)

Stack: Node/TS + Python + RabbitMQ + PostgreSQL + gRPC + Prometheus + K8s

 **Arquitetura:**



Funcionalidades (Semanas 5-6):

- **Payment Service:** REST + gRPC + publisher RabbitMQ
- **Fraud Detection (Python):** consumer RabbitMQ + gRPC server
- **API Gateway:** auth centralizada + rate limiting
- **DLQ:** tratamento de falhas

Circuit Breaker (proteger dependências externas):

```

// Quando Fraud Detection está lento ou fora do ar
class CircuitBreaker {
  state: 'closed' | 'open' | 'half-open' = 'closed';
  failures = 0;
  threshold = 5;
  timeout = 30000; // 30s para half-open

  async call(fn: () => Promise<any>): Promise<any> {
    if (this.state === 'open') {
      throw new CircuitBreakerOpenError('Fraud Detection indisponível');
    }
    try {
      const result = await fn();
      this.reset();
      return result;
    } catch (error) {
      this.failures++;
      if (this.failures >= this.threshold) {
        this.state = 'open';
        setTimeout(() => { this.state = 'half-open'; }, this.timeout);
      }
      throw error;
    }
  }
}

```

Streaming de respostas (file upload, eventos):

```
// Upload de arquivo com streams (sem carregar tudo em memória)
import multer from 'multer';
const upload = multer({ storage: multer.memoryStorage() });

app.post('/upload', upload.single('file'), async (req, res) => {
  // req.file.buffer contém o arquivo → stream para S3
  const uploadStream = uploadToS3(req.file.originalname);
  uploadStream.write(req.file.buffer);
  uploadStream.end();
  uploadStream.on('finish', () => res.json({ status: 'ok' }));
});

// Streaming response (SSE mais robusto)
app.get('/api/reports/stream', async (req, res) => {
  res.writeHead(200, { 'Content-Type': 'application/jsonl' }); // JSON Lines
  const cursor = db.collection('events').find().cursor();
  await cursor.each(async (doc) => {
    res.write(JSON.stringify(doc) + '\n'); // stream linha por linha
  });
  res.end();
});
```

HTTP/2 Multiplexing:

HTTP/1.1: 6 conexões paralelas por domínio, head-of-line blocking
 HTTP/2: multiplexação (várias streams na mesma conexão TCP)
 → sem HOL blocking, push do servidor, compressão de headers

Implementação:

- └─ Nginx: listen 443 ssl http2;
- └─ Node: http2.createServer() em vez de http.createServer()
- └─ gRPC: construído sobre HTTP/2 (multiplexing nativo)

Observabilidade (Semanas 7-8):

```
// Métricas Prometheus
const counter = new Counter({
  name: 'payments_total',
  help: 'Total de pagamentos processados',
  labelNames: ['status'] // approved, denied, error
});

const histogram = new Histogram({
  name: 'payment_duration_seconds',
  help: 'Duração do processamento',
  buckets: [0.1, 0.3, 0.5, 1, 2, 5]
});

// Dashboard Grafana: payments/sec, latency p95, error rate, fraud rate
```

Kubernetes:

```

apiVersion: apps/v1
kind: Deployment
metadata: { name: payment-service }
spec:
  replicas: 3
  selector: { matchLabels: { app: payment-service } }
  template:
    metadata: { labels: { app: payment-service } }
    spec:
      containers:
        - name: payment
          image: seuuser/payment-service:latest
          ports: [{ containerPort: 3001 }]
          envFrom:
            - configMapRef: { name: payment-config }
          livenessProbe: { httpGet: { path: /health, port: 3001 } }
          readinessProbe: { httpGet: { path: /ready, port: 3001 } }
          resources:
            limits: { cpu: "500m", memory: "512Mi" }
            requests: { cpu: "200m", memory: "256Mi" }
---
apiVersion: v1
kind: Service
metadata: { name: payment-service }
spec:
  selector: { app: payment-service }
  ports: [{ port: 3001, targetPort: 3001 }]

```

Checklist: - [] 3 microsserviços + 1 API Gateway - [] RabbitMQ com exchanges + DLQ - [] gRPC entre Payment e Fraud Detection - [] Saga: transação completa, compensação em falha - [] Circuit Breaker no Fraud Detection - [] Métricas Prometheus customizadas - [] Dashboard Grafana - [] K8s manifests (deployments + services) - [] Docker compose para dev local

SEMANA 9 – GraphQL + DataLoader

📖 **Teoria (3h)** - Leia: [Apollo Server Docs](#) - Assista: [GraphQL – N+1 Problem \(DataLoader\)](#)

🔧 **Prática (5h)**

```

const typeDefs = `#graphql
  type Product {
    id: ID!
    name: String!
    price: Float!
    category: Category!
    reviews: [Review!]!
  }
  type Query {
    products(category: String, limit: Int): [Product!]!
    product(id: ID!): Product
  }
  type Mutation {
    createProduct(name: String!, price: Float!, categoryId: ID!): Product!
  }
`;

// DataLoader resolve N+1
const reviewLoader = new DataLoader(async (productIds: number[]) => {
  const reviews = await db.query(
    'SELECT * FROM reviews WHERE product_id = ANY($1)',
    [productIds]
  );
  return productIds.map(id => reviews.filter(r => r.product_id === id));
});

const resolvers = {
  Product: {
    reviews: (parent) => reviewLoader.load(parent.id)
  }
};

```

 **Exercício:** Migre 3 endpoints REST do E-commerce para GraphQL. Implemente DataLoader.

SEMANA 10 – Kubernetes: Probes, HPA, Rollback

 **Teoria (3h)** - Leia: [K8s – Liveness, Readiness, Startup Probes](#) -
Assista: [TechWorld with Nana – K8s Course \(probes + scaling\)](#)

 **Prática (5h)**

```
# HPA - Horizontal Pod Autoscaler
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: payment-service-hpa }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: payment-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70

# Rolling update + rollback
kubectl set image deployment/payment-service payment-service=seuuser/payment-service:v2
kubectl rollout status deployment/payment-service
kubectl rollout undo deployment/payment-service
```

 **Exercício:** Deploy completo do Payment Service no K8s (kind/minikube) com HPA. Faça um rolling update e rollback.

SEMANA 11 – PostgreSQL: Performance e Otimização

 **Teoria (3h)** - Leia: [PostgreSQL Performance Tips](#) - Leia: [Use The Index, Luke](#) – caps 1-4 - Estude: query plan, índices parciais/compostos, vacuum, connection pooling

Prática (5h)

```
-- Analisar performance
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE status = 'pending';

-- Índice parcial (só para pending)
CREATE INDEX CONCURRENTLY idx_orders_pending ON orders(status)
WHERE status IN ('pending', 'processing');

-- Pool de conexões (pgBouncer)
-- ou no app:
const pool = new Pool({ max: 20, idleTimeoutMillis: 30000 });

-- Slow query monitoring
SELECT query, calls, mean_time, rows
FROM pg_stat_statements
ORDER BY mean_time DESC LIMIT 5;

-- Vacuum e Analyze
VACUUM ANALYZE orders;
```

 **Exercício:** Pegue 3 queries do E-commerce/Payments e optimize. Documente o EXPLAIN ANALYZE antes e depois.

SEMANA 12 – Load Testing com k6

 **Teoria (2h)** - Leia: [k6 Docs – Test Types](#) - Estude: smoke, load, stress, spike, soak tests

Prática (5h)

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 50 }, // Ramp up
    { duration: '5m', target: 50 }, // Load
    { duration: '2m', target: 100 }, // Stress
    { duration: '1m', target: 0 }, // Ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<500', 'p(99)<1000'],
    http_req_failed: ['rate<0.01'],
  },
};

export default function () {
  const res = http.post('http://localhost:3000/api/payments', JSON.stringify({
    orderId: '123', amount: 250.00
  }), { headers: { 'Content-Type': 'application/json' } });

  check(res, {
    'status 201': (r) => r.status === 201,
    'response < 300ms': (r) => r.timings.duration < 300,
  });
  sleep(1);
}
```

 **Exercício:** Teste o Payment Service com 50, 100, 200 VUs. Identifique o bottleneck. Documente com gráficos.

Backpressure + Connection Pooling na prática:

Connection Pooling (PostgreSQL):

- └ Pool fixo: max 20 conexões → requests excedentes esperam
- └ Pool dinâmico: min 5, max 50 → escala conforme demanda
- └ Cuidado: cada conexão usa ~10MB de RAM (max 500 = 5GB)
- └ pgBouncer: pooler externo, modo transaction (mais eficiente)

Backpressure na API:

- └ req.on('data', (chunk) => { ... }) → readable.resume() / readable.pause()
- └ Limite de body: app.use(express.json({ limit: '10mb' }))
- └ Rate limiting por IP + global
- └ Circuit breaker para chamadas externas (já implementado)

SEMANAS 13-14 – 🚀 Mini Projeto: Servidor HTTP em C++

Stack: C++17, POSIX sockets, threads

Conceitos: Thread pool, connection pooling, parsing HTTP, benchmark

Funcionalidades:

```
// Servidor HTTP/1.1 multi-thread com thread pool
class ThreadPool {
    vector<thread> workers;
    queue<function<void()>> tasks;
    mutex queue_mutex;
    condition_variable condition;
    bool stop;

public:
    ThreadPool(size_t threads) { /* cria N threads que ficam esperando tarefas */ }
    template<class F> void enqueue(F&& f) { /* adiciona tarefa à fila */ }
    ~ThreadPool() { /* join em todas as threads */ }
};

// Thread principal aceita conexões e enfileira
while (true) {
    int client_fd = accept(server_fd, nullptr, nullptr);
    pool.enqueue([client_fd] {
        handle_client(client_fd);
        close(client_fd);
    });
}

// Parse de requisição
void handle_client(int fd) {
    char buffer[4096];
    read(fd, buffer, 4096);
    string request(buffer);
    auto method = request.substr(0, request.find(' '));
    auto path = request.substr(request.find(' ') + 1);
    path = path.substr(0, path.find(' '));
    // ... responde com HTTP/1.1 200 OK
}
```

Benchmark vs Node.js:

```
k6 run --vus 100 --duration 30s loadtest.js
# Compare: req/s, latency p95, erro rate
```

Checklist: - [] Single-thread funcional - [] Thread pool com tamanho configurável - [] Parse de requisições (GET, POST) - [] Respostas HTTP/1.1 - [] Benchmark vs Node.js - [] CMakeLists.txt - [] README com resultados

SEMANAS 15-16 – Documentação, Revisão e LeetCode

Semana 15: - Documentação de arquitetura dos projetos (ARCHITECTURE.md) - README profissional com badges - Resolva 5 LeetCode Médio (foco em árvores e grafos)

Semana 16 –  Prova de Nível:

1. **RabbitMQ:** Diferença entre exchanges direct, topic, fanout. Dê exemplos de uso
2. **Event-Driven:** Vantagens de saga coreografada vs orquestrada?
3. **Microserviços:** Comunicação síncrona vs assíncrona – trade-offs?
4. **gRPC:** Por que Protocol Buffers é mais rápido que JSON? Limitações?
5. **K8s:** Explique liveness vs readiness probe. O que acontece se cada uma falhar?
6. **C++:** Compare thread pool vs criar thread por requisição. Impacto na performance?
7. **DB:** O que é um índice parcial? Em qual cenário é útil?

 **Passa se:** Responde $\geq 5/7$ + Sistema de Pagamentos completo + C++ server funcional.

SEMANAS 17-20 – Projeto Final + Estudo Dirigido

Semana	Atividade
17	Refatorar Payment Service (adicionar OpenTelemetry tracing)
18	Configurar Loki + Promtail para logs centralizados
19	Implementar health checks + graceful shutdown no C++ server
20	Revisão geral + preparação para entrevistas (System Design)



Resumo do Nível

Tecnologia	O que você domina
RabbitMQ	Exchanges, DLQ, RPC, publisher confirms
Event-Driven	Saga coreografada, compensação, consistência eventual
gRPC	Unary + streaming, protobuf, performance
GraphQL	Schema, resolvers, DataLoader, subscriptions
Microserviços	Gateway, service discovery, comunicação síncrona/assíncrona
K8s	Deployments, services, probes, HPA, rolling update
Observabilidade	Prometheus, Grafana, métricas customizadas
C++	Thread pool, sockets, servidor HTTP, benchmark
PostgreSQL	Query plan, índices parciais, pool, profiling

Próximo nível: `level-04-mastery.md`